Experiment No 1

To implement Principal Component Analysis (PCA) for dimensionality reduction and transform wine dataset features into a smaller number of principal components for visualization and analysis.

Aim

To apply PCA to the Wine dataset, reduce its dimensionality, calculate the explained variance of principal components, and visualize the transformed samples to inspect the separation of wine classes.

Software Requirements

- Python 3.x
- Jupyter Notebook / JupyterLab / Google Colab
- NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn

Install the required libraries if they are not already available:

```
!pip install numpy pandas matplotlib seaborn scikit-learn
```

Theory

Dimensionality Reduction

Dimensionality reduction reduces the number of input features while retaining as much useful information as possible. It simplifies analysis and visualization and can reduce computational cost. The Wine dataset contains numerical features such as Alcohol, Malic Acid, Ash, Magnesium, Total Phenols, Flavanoids, Color Intensity, Hue, OD280, and Proline.

Principal Component Analysis

Principal Component Analysis (PCA) is an unsupervised dimensionality-reduction technique. It transforms the original variables into new, mutually orthogonal (and therefore uncorrelated) variables called principal components. PC1 captures the maximum possible variance; PC2 captures the maximum remaining variance subject to being orthogonal to PC1, and so on.

Feature Standardization

Wine features use different numerical scales. Standardization is applied before PCA so that variables with large magnitudes do not dominate the analysis.

$Z = (X - \mu) / \sigma$

Here, X is the original value, μ is the feature mean, σ is its standard deviation, and Z is the standardized value.

Dataset Description

The code assumes that Wine.csv has numerical input features and a class column named Customer_Segment. If your CSV uses a different target-column name, update target_column in the code. Confirm the actual dimensions and columns using df.shape and df.columns.

No.	Feature	Description
1	Alcohol	Alcohol content
2	Malic_Acid	Malic acid

3	Ash	Ash content
4	Ash_Alcanity	Ash alkalinity
5	Magnesium	Magnesium content
6	Total_Phenols	Total phenols
7	Flavanoids	Flavanoid content
8	Nonflavanoid_Phenols	Nonflavanoid phenols
9	Proanthocyanins	Proanthocyanins
10	Color_Intensity	Color intensity
11	Hue	Hue
12	OD280	OD280 measurement
13	Proline	Proline content

The class column is not used to calculate PCA. It is retained only to color or label the PCA plot and inspect class separation.

Procedure

1. Import the required Python libraries.
2. Load Wine.csv and inspect its shape, columns, data types, and missing values.
3. Separate the numerical input features from the target/class column.
4. Standardize the input features using StandardScaler.
5. Apply PCA to all standardized features.
6. Calculate explained variance ratio and cumulative explained variance.
7. Determine the number of components needed to retain 90% or 95% variance.
8. Transform the dataset to two principal components for visualization.
9. Plot PC1 against PC2, using class labels only for visualization.
10. Interpret variance results and the degree of visual class separation.

Conclusion:

Code

cell 1 – Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

import warnings
warnings.filterwarnings("ignore")
```

Cell 2 – Load the Dataset

```
# Keep Wine.csv in the same folder as this notebook
df = pd.read_csv("Wine.csv")

display(df.head())
print("Dataset shape:", df.shape)
print("Columns:", df.columns.tolist())
```

Cell 3 – Inspect the Dataset

```
print(df.info())
print("\nMissing values in each column:")
print(df.isnull().sum())

display(df.describe())
```

Cell 4 – Separate Features and Target

```
target_column = "Customer_Segment"

# If your CSV uses a different class-column name, change target_column.
X = df.drop(columns=[target_column])
y = df[target_column]

print("Input shape:", X.shape)
print("Target shape:", y.shape)
```

Cell 5 – Standardize the Features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
display(X_scaled_df.head())
```

```
print("Feature means (approximately zero):")
print(X_scaled_df.mean().round(2))

print("\nFeature standard deviations (approximately one):")
print(X_scaled_df.std(ddof=0).round(2))
```

Cell 6 – Apply PCA to All Features

```
pca = PCA()
X_pca = pca.fit_transform(X_scaled)

print("Original feature shape:", X.shape)
print("PCA-transformed shape:", X_pca.shape)
```

Cell 7 – Explained Variance Table

```
explained_variance = pca.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

variance_df = pd.DataFrame({
    "Principal Component": [f"PC{i}" for i in range(1, len(explained_variance) + 1)],
    "Explained Variance (%)": explained_variance * 100,
    "Cumulative Variance (%)": cumulative_variance * 100
})

display(variance_df.round(2))
```

Cell 8 – Plot Explained and Cumulative Variance

```
plt.figure(figsize=(9, 5))
plt.plot(range(1, len(explained_variance) + 1),
         explained_variance * 100, marker="o")
plt.xlabel("Principal Component")
plt.ylabel("Explained Variance (%)")
plt.title("Explained Variance by Principal Component")
plt.xticks(range(1, len(explained_variance) + 1))
plt.grid(True)
plt.show()

plt.figure(figsize=(9, 5))
plt.plot(range(1, len(cumulative_variance) + 1),
         cumulative_variance * 100, marker="o")
plt.axhline(y=90, linestyle="--", label="90% variance")
plt.axhline(y=95, linestyle=":", label="95% variance")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance (%)")
```

```
plt.title("Cumulative Explained Variance")
plt.xticks(range(1, len(cumulative_variance) + 1))
plt.legend()
plt.grid(True)
plt.show()
```

Cell 9 – Determine the Number of Components

```
n_90 = np.argmax(cumulative_variance >= 0.90) + 1
n_95 = np.argmax(cumulative_variance >= 0.95) + 1

print("Components needed for at least 90% variance:", n_90)
print("Variance retained:", round(cumulative_variance[n_90 - 1] * 100, 2), "%")

print("\nComponents needed for at least 95% variance:", n_95)
print("Variance retained:", round(cumulative_variance[n_95 - 1] * 100, 2), "%")
```

Cell 10 – Transform Data to Two Components

```
pca_2 = PCA(n_components=2)
X_pca_2 = pca_2.fit_transform(X_scaled)

pca_df = pd.DataFrame(X_pca_2, columns=["PC1", "PC2"])
pca_df[target_column] = y.to_numpy()

print("Original number of features:", X.shape[1])
print("Reduced number of features:", X_pca_2.shape[1])
print("Transformed shape:", X_pca_2.shape)

print("PC1 variance:", round(pca_2.explained_variance_ratio_[0] * 100, 2), "%")
print("PC2 variance:", round(pca_2.explained_variance_ratio_[1] * 100, 2), "%")
print("Combined variance:", round(pca_2.explained_variance_ratio_.sum() * 100, 2), "%")

display(pca_df.head())
```

Cell 11 – PCA Scatter Plot

```
plt.figure(figsize=(9, 6))
sns.scatterplot(
    data=pca_df,
    x="PC1",
    y="PC2",
    hue=target_column,
    palette="deep",
    s=75
)
plt.title("PCA Projection of the Wine Dataset")
```

```
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.legend(title="Wine Class")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Cell 12 – PCA Loadings

```
loadings = pd.DataFrame(
    pca_2.components_.T,
    index=X.columns,
    columns=["PC1", "PC2"]
)

display(loadings)

print("Largest absolute loadings for PC1:")
display(loadings["PC1"].abs().sort_values(ascending=False))

print("Largest absolute loadings for PC2:")
display(loadings["PC2"].abs().sort_values(ascending=False))

loadings["PC1"].sort_values().plot(kind="barh", figsize=(8, 6),
                                   title="Feature Loadings for PC1")
plt.xlabel("Loading")
plt.tight_layout()
plt.show()
```

9. Complete Code in One Cell

The following compact version performs the complete PCA workflow. Use either the step-by-step cells above or this single-cell version.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

df = pd.read_csv("Wine.csv")
target_column = "Customer_Segment"  # Update if needed

print("Dataset shape:", df.shape)
print("Missing values:\n", df.isnull().sum())
```

```
X = df.drop(columns=[target_column])
y = df[target_column]

X_scaled = StandardScaler().fit_transform(X)

pca = PCA()
X_pca = pca.fit_transform(X_scaled)
evr = pca.explained_variance_ratio_
cum_evr = np.cumsum(evr)

variance_df = pd.DataFrame({
   "PC": [f"PC{i}" for i in range(1, len(evr) + 1)],
   "Explained Variance (%)": evr * 100,
   "Cumulative Variance (%)": cum_evr * 100
})
display(variance_df.round(2))

plt.figure(figsize=(9, 5))
plt.plot(range(1, len(cum_evr) + 1), cum_evr * 100, marker="o")
plt.axhline(90, linestyle="--", label="90%")
plt.axhline(95, linestyle=":", label="95%")
plt.xlabel("Number of Principal Components")
plt.ylabel("Cumulative Explained Variance (%)")
plt.title("Cumulative Explained Variance")
plt.legend()
plt.grid(True)
plt.show()

pca2 = PCA(n_components=2)
X2 = pca2.fit_transform(X_scaled)

plot_df = pd.DataFrame(X2, columns=["PC1", "PC2"])
plot_df[target_column] = y.to_numpy()

print("Original features:", X.shape[1])
print("Reduced features:", X2.shape[1])
print("Variance retained by PC1 and PC2:",
     round(pca2.explained_variance_ratio_.sum() * 100, 2), "%")

plt.figure(figsize=(9, 6))
sns.scatterplot(data=plot_df, x="PC1", y="PC2",
                hue=target_column, s=75, palette="deep")
```

```
plt.title("PCA Projection of the Wine Dataset")
plt.grid(True)
plt.tight_layout()
plt.show()
```

Post Lab Viva Questions

1. What is Principal Component Analysis (PCA), and why is it used for dimensionality reduction?
2. Why is feature standardization necessary before applying PCA to the Wine dataset?
3. What is the difference between original features and principal components?
4. What does the explained variance ratio represent in PCA?
5. Why are PC1 and PC2 commonly used to visualize high-dimensional datasets?
6. How does PCA help identify patterns or separation among different wine classes?
7. Why is PCA considered an unsupervised learning technique?

Experiment No 2

Predict the price of the Uber ride from a given pickup point to the agreed drop-off location.

Aim

To predict the price of an Uber ride based on pickup and drop-off locations by performing data preprocessing, outlier detection, correlation analysis, and implementing Linear Regression, Ridge Regression, and Lasso Regression models. Evaluate and compare the models using R^2 score, RMSE, MAE, and Median Absolute Error.

- **Software Requirements**

Software requirements

- Python 3.x
- Jupyter Notebook or Google Colab
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn

Dataset Description

The experiment uses an Uber fare dataset, typically stored in a CSV file named uber.csv.

Attribute	Description
fare_amount	Fare charged for the ride; target variable
pickup_longitude	Longitude of pickup location
pickup_latitude	Latitude of pickup location
dropoff_longitude	Longitude of drop-off location
dropoff_latitude	Latitude of drop-off location
passenger_count	Number of passengers
trip_distance_km	Calculated trip distance in kilometres

Theory

Data Preprocessing

Data preprocessing includes checking data types, removing duplicates, handling missing values, validating coordinates and fare values, and creating useful features.

Outlier Detection

The Interquartile Range method uses IQR = Q3 − Q1. Potential outliers lie below Q1 − 1.5(IQR) or above Q3 + 1.5(IQR). Domain knowledge should be used before removing observations.

Correlation

Correlation measures the strength and direction of a linear relationship. Values near +1 indicate strong positive correlation, values near −1 strong negative correlation, and values near 0 weak linear correlation.

Linear Regression

Multiple Linear Regression represents the prediction as: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_p x_p$.

Ridge Regression

Ridge Regression adds an L2 penalty to the objective function and shrinks large coefficients.

Lasso Regression

Lasso Regression adds an L1 penalty and can shrink some coefficients to zero, providing feature selection.

Evaluation Metrics

- R^2 Score: proportion of target variance explained; higher is generally better.
- RMSE: square root of mean squared error; lower is better and large errors are penalized more.
- MAE: average absolute prediction error; lower is better.
- MSE: average squared prediction error; lower is better.

Procedure / Algorithm

i. Import required Python libraries.
ii. Load the Uber CSV dataset.
iii. Inspect shape, columns, data types, missing values and statistics.
iv. Remove duplicates and invalid fare values.
v. Validate pickup and drop-off coordinates.
vi. Calculate trip distance using the Haversine formula.
vii. Inspect fare distribution and potential outliers.
viii. Generate a correlation matrix and heatmap.
ix. Select input features and target variable.
x. Split data into training and testing sets.
xi. Train Linear Regression, Ridge Regression and Lasso Regression.
xii. Calculate R^2, RMSE, MAE and MSE.
xiii. Compare model performance and record observations.

Conclusion:

Program:

Cell 1 – Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

df = pd.read_csv("uber.csv")
print("Dataset shape:", df.shape)
display(df.head())
```

Cell 2 – Preprocessing

```
df.columns = [c.strip().lower().replace(" ", "_") for c in df.columns]
df.drop_duplicates(inplace=True)

numeric_cols = ["fare_amount","pickup_longitude","pickup_latitude",
                "dropoff_longitude","dropoff_latitude","passenger_count"]

for col in numeric_cols:
    if col in df.columns:
        df[col] = pd.to_numeric(df[col], errors="coerce")

df = df[df["fare_amount"].notna() & (df["fare_amount"] > 0)].copy()

print("Shape after preprocessing:", df.shape)
print(df.isnull().sum())
```

Cell 3 – Haversine Distance

```
def haversine_distance(lat1, lon1, lat2, lon2):
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = (np.sin(dlat/2)**2 +
         np.cos(lat1)*np.cos(lat2)*np.sin(dlon/2)**2)
    return 6371 * 2 * np.arcsin(np.sqrt(np.clip(a, 0, 1)))
```

```
coord_cols = ["pickup_longitude","pickup_latitude",
              "dropoff_longitude","dropoff_latitude"]

if all(c in df.columns for c in coord_cols):
    valid = (
        df["pickup_longitude"].between(-180,180) &
        df["dropoff_longitude"].between(-180,180) &
        df["pickup_latitude"].between(-90,90) &
        df["dropoff_latitude"].between(-90,90) &
        (df["pickup_longitude"] != 0) &
        (df["dropoff_longitude"] != 0) &
        (df["pickup_latitude"] != 0) &
        (df["dropoff_latitude"] != 0)
    )
    df = df[valid].copy()
    df["trip_distance_km"] = haversine_distance(
        df["pickup_latitude"], df["pickup_longitude"],
        df["dropoff_latitude"], df["dropoff_longitude"])

print("Shape after coordinate validation:", df.shape)
display(df.head())
```

Cell 4 – Outlier Identification

```
plt.figure(figsize=(8,4))
sns.histplot(df["fare_amount"], bins=50, kde=True)
plt.title("Fare Distribution")
plt.show()

plt.figure(figsize=(8,3))
sns.boxplot(x=df["fare_amount"])
plt.title("Fare Outlier Detection")
plt.show()

Q1 = df["fare_amount"].quantile(0.25)
Q3 = df["fare_amount"].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers = df[(df["fare_amount"] < lower) | (df["fare_amount"] > upper)]
print("Q1:",Q1,"Q3:",Q3,"IQR:",IQR)
print("Lower limit:",lower,"Upper limit:",upper)
print("Potential outliers:",len(outliers))
```

```
# After inspection, you may remove them:
# df = df[df["fare_amount"].between(lower, upper)].copy()
```

Cell 5 – Correlation Analysis

```
numeric_df = df.select_dtypes(include=np.number)
corr = numeric_df.corr()

plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", center=0)
plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()

print("Correlation with fare_amount:")
display(corr["fare_amount"].drop("fare_amount").sort_values(ascending=False))
```

Cell 6 – Feature Selection and Split

```
features = ["pickup_longitude","pickup_latitude",
            "dropoff_longitude","dropoff_latitude",
            "passenger_count","trip_distance_km"]
features = [c for c in features if c in df.columns]

X = df[features].replace([np.inf,-np.inf],np.nan)
X = X.dropna(axis=1, how="all")
y = df["fare_amount"]

X_train,X_test,y_train,y_test = train_test_split(
   X,y,test_size=0.20,random_state=42)

print("Features:",X.columns.tolist())
print("Training samples:",len(X_train))
print("Testing samples:",len(X_test))
```

Cell 7 – Train Models

```
def make_pipeline(model):
   return Pipeline([
      ("imputer", SimpleImputer(strategy="median")),
      ("scaler", StandardScaler()),
      ("regressor", model)
   ])

models = {
   "Linear Regression": make_pipeline(LinearRegression()),
   "Ridge Regression": make_pipeline(Ridge(alpha=1.0)),
```

```
    "Lasso Regression": make_pipeline(Lasso(alpha=0.1, max_iter=20000))
}

predictions = {}
for name, model in models.items():
    model.fit(X_train,y_train)
    predictions[name] = model.predict(X_test)
    print(name,"trained successfully")
```

Cell 8 – Evaluate Models

```
results = []

for name,y_pred in predictions.items():
    results.append({
        "Model":name,
        "R2 Score":r2_score(y_test,y_pred),
        "RMSE":np.sqrt(mean_squared_error(y_test,y_pred)),
        "MAE":mean_absolute_error(y_test,y_pred),
        "MSE":mean_squared_error(y_test,y_pred)
    })

results_df = pd.DataFrame(results).sort_values("RMSE")
display(results_df.round(4))
print("Best model based on RMSE:",results_df.iloc[0]["Model"])
```

Cell 9 – Actual vs Predicted

```
for name,y_pred in predictions.items():
    plt.figure(figsize=(6,5))
    plt.scatter(y_test,y_pred,alpha=0.4)
    mn=min(y_test.min(),y_pred.min())
    mx=max(y_test.max(),y_pred.max())
    plt.plot([mn,mx],[mn,mx],"r--")
    plt.xlabel("Actual Fare")
    plt.ylabel("Predicted Fare")
    plt.title(name+" - Actual vs Predicted")
    plt.tight_layout()
    plt.show()
```

Cell 10 – New Ride Prediction

```
new_ride = {
    "pickup_longitude":-73.9857,
    "pickup_latitude":40.7484,
    "dropoff_longitude":-73.9851,
    "dropoff_latitude":40.7580,
```

```
    "passenger_count":1
}

if "trip_distance_km" in X.columns:
    new_ride["trip_distance_km"] = haversine_distance(
        new_ride["pickup_latitude"],new_ride["pickup_longitude"],
        new_ride["dropoff_latitude"],new_ride["dropoff_longitude"])

new_ride_df = pd.DataFrame([new_ride]).reindex(columns=X.columns)

for name,model in models.items():
    fare = model.predict(new_ride_df)[0]
    print(f"{name}: Predicted fare = {fare:.2f}")
```

Post-Lab Questions

1. What is the purpose of preprocessing the Uber fare dataset?
2. Explain the IQR method for identifying outliers.
3. What is the purpose of calculating Haversine distance?
4. What does a correlation coefficient of +1, −1 and 0 indicate?
5. Write the equation of multiple Linear Regression.
6. Differentiate between Linear Regression and Ridge Regression.
7. How does Lasso Regression perform feature selection?
8. Why is feature scaling important for Ridge and Lasso Regression?
9. Differentiate between R^2, RMSE, MAE and MSE.

Experiment No 3

Implement K-Nearest Neighbours' algorithm on Social network ad dataset. Compute confusion matrix, accuracy, error rate, precision and recall on the given dataset.

Aim:

To implement the K-Nearest Neighbours (KNN) classification algorithm on the Social Network Ads dataset and compute the confusion matrix, accuracy, error rate, precision and recall.

Dataset Description

The Social Network Ads dataset is commonly used as a binary classification dataset. Typical attributes include User ID, Gender, Age, EstimatedSalary and Purchased. The target variable is Purchased, where 0 and 1 represent the two classes.

Attribute	Description	Role
User ID	Unique identifier of a user	Identifier; normally excluded
Gender	Gender category	Categorical input; can be encoded
Age	Age of the user	Input feature
Estimated Salary	Estimated salary	Input feature
Purchased	Whether the user purchased the product	Target

Theory

KNN is a supervised, instance-based learning algorithm. For a new sample, the algorithm calculates its distance from training samples, selects the K nearest samples, and assigns the class occurring most frequently among those neighbours. Euclidean distance is commonly used for continuous features.

Because KNN is distance-based, feature scaling is important. Standardization transforms a feature approximately as $z = (x - \text{mean}) / \text{standard deviation}$, preventing a feature with a larger numerical range from dominating the distance calculation.

Performance Measures

Measure	Expression / Meaning
Confusion Matrix	Shows counts of TP, TN, FP and FN.
Accuracy	(TP + TN) / (TP + TN + FP + FN)
Error Rate	1 − Accuracy
Precision	TP / (TP + FP)
Recall	TP / (TP + FN)

Algorithm

1. Import the required Python libraries.
2. Load the Social Network Ads dataset.
3. Inspect missing values, duplicate records and data types.
4. Encode the categorical Gender attribute if it is present.
5. Separate the input features X and target y.
6. Split the data into training and testing subsets.

7. Standardize the numerical input features.
8. Create a KNN classifier with a selected value of K.
9. Train the model using the training data.
10. Predict the target class for the testing data.
11. Generate the confusion matrix and calculate accuracy, error rate, precision and recall.
12. Interpret the evaluation results.

Conclusion:

Program:

```
 # Experiment No. 3: KNN on Social Network Ads Dataset

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    confusion_matrix, accuracy_score, precision_score, recall_score,
    classification_report
)

# 1. Load dataset
df = pd.read_csv("Social_Network_Ads.csv")

print("Dataset Preview:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nMissing Values:")
print(df.isnull().sum())

# 2. Remove duplicate records
df = df.drop_duplicates().reset_index(drop=True)

# 3. Encode Gender, if present
if "Gender" in df.columns:
    df["Gender"] = df["Gender"].map({"Male": 1, "Female": 0})

# 4. Define input and target
# User ID is an identifier, so it is not used as a predictive feature.
X = df.drop(columns=["Purchased", "User ID"], errors="ignore")
y = df["Purchased"]

# Keep numeric columns only after encoding categorical data
X = X.select_dtypes(include=[np.number])

# 5. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

# 6. Feature scaling
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)

# 7. Build and train KNN model
k = 5
knn = KNeighborsClassifier(n_neighbors=k)
knn.fit(X_train_scaled, y_train)

# 8. Prediction
y_pred = knn.predict(X_test_scaled)

# 9. Evaluation
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1 - accuracy
precision = precision_score(y_test, y_pred, zero_division=0)
recall = recall_score(y_test, y_pred, zero_division=0)

print("\nConfusion Matrix:")
print(cm)

print("\nAccuracy :", accuracy)
print("Error Rate:", error_rate)
print("Precision:", precision)
print("Recall   :", recall)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, zero_division=0))

# 10. Display confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False)
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("KNN Confusion Matrix")
plt.show()
```

Post-Lab Questions

1. What is the basic principle of KNN?
2. Why is feature scaling important for KNN?
3. What is the effect of increasing or decreasing K?
4. Why is User ID generally excluded from model training?
5. Differentiate precision and recall.
6. How is error rate related to accuracy?
7. What does a confusion matrix represent?
8. What is the difference between supervised and unsupervised learning?

Experiment No 4

Implement K-Means clustering on Iris.csv dataset. Determine the number of clusters using the elbow method.

Aim:

To implement the K-Means clustering algorithm on the Iris.csv dataset and determine the suitable number of clusters using the Elbow Method.

Objectives

- To understand unsupervised learning and K-Means clustering.
- To preprocess and standardize Iris feature data.
- To calculate within-cluster sum of squares (WCSS/inertia) for different values of K.
- To determine the appropriate number of clusters using the elbow curve.
- To visualize the resulting clusters and compare them with the actual species labels.

Dataset Description

The Iris dataset contains measurements of iris flowers. The commonly used Iris.csv file contains Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm and Species. In clustering, Id and Species are excluded from the feature matrix; Species may be retained only for comparison/visualization after clustering.

Feature	Description
SepalLengthCm	Sepal length in centimetres
SepalWidthCm	Sepal width in centimetres
PetalLengthCm	Petal length in centimetres
PetalWidthCm	Petal width in centimetres
Species	Actual species label; not used to train K-Means

Theory

K-Means is an unsupervised clustering algorithm that partitions observations into K clusters. It begins with K centroids, assigns each sample to the nearest centroid, updates each centroid as the mean of the assigned samples, and repeats these steps until the assignments/centroids stabilize or the iteration limit is reached.

The Elbow Method evaluates several K values and plots WCSS (also called inertia) against K. WCSS decreases as K increases. The point where the reduction in WCSS begins to diminish substantially is treated as the elbow and is selected as a practical value of K.

Algorithm

1. Import Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn libraries.
2. Load Iris.csv.
3. Inspect the first records and dataset information.
4. Generate summary statistics and feature visualizations.
5. Remove Id and Species from the clustering feature matrix.
6. Standardize the numerical features using StandardScaler.

7. Run K-Means for K values from 1 to 10 and store the inertia/WCSS values.
8. Plot K against WCSS and identify the elbow.
9. Set the optimal K. For the supplied notebook code, K = 3 is used.
10. Fit K-Means with K = 3 and obtain cluster labels.
11. Visualize the actual species distribution and K-Means cluster distribution.
12. Interpret the clustering result.

Conclusion:

Post Lab Questions:

1. What type of learning is K-Means?
2. What does K represent in K-Means?
3. Explain the Elbow Method.
4. Why is standardization useful before clustering?
5. Can K-Means work with categorical data directly?
6. What happens if K is chosen too large?
7. What is the difference between clustering and classification?

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Set plot style
sns.set_theme(style="whitegrid")

# Load the dataset
df = pd.read_csv('Iris.csv')

# Display the first 5 rows
print("Dataset Preview:")
display(df.head())

# Check dataset information
print("\nDataset Info:")
df.info()

# Summary statistics
print(df.describe())

# Pairplot to visualize feature relationships colored by true species
sns.pairplot(df.drop(columns=['Id'], errors='ignore'), hue='Species', palette='Set2')
plt.show()

# Extract features (drop Id and Species columns)
X = df.drop(columns=['Id', 'Species'], errors='ignore')

# Standardize features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("Scaled features shape:", X_scaled.shape)

# Calculate inertia for K = 1 to 10
inertia = []
K_range = range(1, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)

# Plotting the Elbow Curve
plt.figure(figsize=(8, 5))
```

```
plt.plot(K_range, inertia, marker='o', linestyle='-', color='b',
         linewidth=2, markersize=8)
plt.title('The Elbow Method for Optimal K', fontsize=14, fontweight='bold')
plt.xlabel('Number of Clusters (K)', fontsize=12)
plt.ylabel('Inertia (WCSS)', fontsize=12)
plt.xticks(K_range)
plt.grid(True)
plt.show()

# Initialize K-Means with optimal clusters
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(X_scaled)

# Add cluster labels to dataframe
df['Cluster'] = cluster_labels

# View cluster centers in scaled space
print("Cluster Centers (Scaled):\n", kmeans.cluster_centers_)

# Actual Species Plot and K-Means Clusters Plot
fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharex=True, sharey=True)

# Actual Species Plot
sns.scatterplot(
    data=df, x='PetalLengthCm', y='PetalWidthCm', hue='Species',
    palette='Set1', s=70, ax=axes[0]
)
axes[0].set_title('Actual Species Distribution', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Petal Length (cm)')
axes[0].set_ylabel('Petal Width (cm)')

# K-Means Clusters Plot
sns.scatterplot(
    data=df, x='PetalLengthCm', y='PetalWidthCm', hue='Cluster',
    palette='viridis', s=70, ax=axes[1], legend='full'
)
axes[1].set_title('K-Means Clusters (K=3)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Petal Length (cm)')
axes[1].set_ylabel('Petal Width (cm)')

plt.tight_layout()
plt.show()
```

Experiment No 5

Implement Random Forest Classifier model to predict the safety of the car

Aim:

To implement a Random Forest Classifier for predicting the safety/evaluation class of a car using the Car Evaluation dataset.

Dataset Description

The attached car_evaluation.csv file contains 1,727 data rows and 7 columns. The file does not contain a header row; its first row is a data record. The standard Car Evaluation attributes can therefore be assigned as: buying, maint, doors, persons, lug_boot, safety and class.

Attribute	Meaning / Typical Values
buying	Buying price: vhigh, high, med, low
maint	Maintenance price: vhigh, high, med, low
doors	Number of doors: 2, 3, 4, 5more
persons	Passenger capacity: 2, 4, more
lug_boot	Luggage boot size: small, med, big
safety	Safety: low, med, high
class	Car evaluation class: unacc, acc, good, vgood

Theory

Random Forest is an ensemble supervised learning algorithm that builds multiple decision trees and combines their predictions. For classification, the final class is generally selected by majority voting among the trees. Randomness is introduced through bootstrapped training samples and random subsets of features considered at tree splits. The combination of multiple trees generally improves robustness compared with a single decision tree.

Algorithm

1. Import Pandas, NumPy, Matplotlib, Seaborn and Scikit-learn modules.
2. Load car_evaluation.csv without assuming that its first row is a header.
3. Assign the seven standard Car Evaluation attribute names.
4. Check the dataset for missing values and duplicates.
5. Separate the input attributes from the target class.
6. Encode categorical attributes numerically.
7. Split the data into training and testing sets.
8. Create the Random Forest Classifier.
9. Train the classifier on the training set.
10. Predict the test-set classes.
11. Calculate accuracy and generate the confusion matrix and classification report.
12. Interpret the predicted safety/evaluation classes.

Conclusion:

Program:

```
# Experiment No. 5: Random Forest Classifier for Car Safety Prediction

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, confusion_matrix, classification_report
)

# 1. Load the attached headerless dataset
file_path = "car_evaluation.csv"

columns = [
    "buying", "maint", "doors", "persons",
    "lug_boot", "safety", "class"
]

df = pd.read_csv(file_path, header=None, names=columns)

print("Dataset Shape:", df.shape)
print("\nDataset Preview:")
print(df.head())

print("\nMissing Values:")
print(df.isnull().sum())

# 2. Remove duplicate records, if any
df = df.drop_duplicates().reset_index(drop=True)

# 3. Separate input and target
X = df.drop(columns=["class"])
y = df["class"]

# 4. Encode categorical columns
X_encoded = pd.DataFrame(index=X.index)

encoders = {}
for col in X.columns:
    le = LabelEncoder()
    X_encoded[col] = le.fit_transform(X[col].astype(str))
    encoders[col] = le

# Encode target
```

```
target_encoder = LabelEncoder()
y_encoded = target_encoder.fit_transform(y.astype(str))

print("\nEncoded Features:")
print(X_encoded.head())

# 5. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_encoded, y_encoded,
    test_size=0.25,
    random_state=42,
    stratify=y_encoded
)

# 6. Random Forest model
rf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

# 7. Training
rf.fit(X_train, y_train)

# 8. Prediction
y_pred = rf.predict(X_test)

# 9. Evaluation
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print("\nAccuracy:", accuracy)
print("\nConfusion Matrix:")
print(cm)

print("\nClassification Report:")
print(
    classification_report(
        y_test, y_pred,
        target_names=target_encoder.classes_,
        zero_division=0
    )
)

# 10. Confusion matrix visualization
plt.figure(figsize=(7, 6))
sns.heatmap(
    cm, annot=True, fmt="d", cmap="Blues", cbar=False,
    xticklabels=target_encoder.classes_,
    yticklabels=target_encoder.classes_
)
```

```
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Random Forest - Car Evaluation Confusion Matrix")
plt.show()

# 11. Feature importance
importance = pd.Series(
    rf.feature_importances_,
    index=X_encoded.columns
).sort_values(ascending=False)

plt.figure(figsize=(8, 5))
importance.plot(kind="bar")
plt.ylabel("Importance")
plt.xlabel("Feature")
plt.title("Random Forest Feature Importance")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Post Lab Questions:

1. What is ensemble learning?
2. How does Random Forest differ from a single Decision Tree?
3. What is bootstrapping in Random Forest?
4. What is meant by feature randomness?
5. Why are categorical variables encoded before model training?
6. What is feature importance?
7. What is overfitting and how can Random Forest reduce it?

Experiment No 6

Build a Tic-Tac-Toe game using reinforcement learning in Python by using following tasks a. Setting up the environment b. Defining the Tic-Tac-Toe game c. Building the reinforcement learning model d. Training the model e. Testing the model

Aim:

To build a Tic-Tac-Toe game using reinforcement learning in Python by setting up the environment, defining the game, building the reinforcement learning model, training the model and testing the trained agent.

Reinforcement Learning Concepts

Term	Tic-Tac-Toe Interpretation
Environment	The Tic-Tac-Toe board and game rules
State	Current arrangement of X, O and empty cells
Action	Selecting an empty cell
Reward	Positive for winning, negative for losing, small/zero for draw or intermediate move
Policy	Strategy used by the agent to select actions
Q-value	Estimated future return for taking an action in a state
Episode	One complete game from start to terminal state

Theory: Q-Learning

Q-learning is a model-free reinforcement learning algorithm. It learns an action-value function Q(s,a), which estimates the expected cumulative reward obtained by taking action a in state s and following the learned policy thereafter.

A standard update rule is: $Q(s,a) \leftarrow Q(s,a) + \alpha\,[r + \gamma \max(Q(s',a')) - Q(s,a)]$, where α is the learning rate, γ is the discount factor, r is the immediate reward and s' is the next state.

During training, the agent balances exploration and exploitation. With an ε-greedy strategy, the agent chooses a random legal action with probability ε and otherwise chooses the action having the highest Q-value. ε can be gradually reduced during training.

Tasks

1. Setting up the environment.
2. Defining the Tic-Tac-Toe game.
3. Building the reinforcement learning model.
4. Training the model.
5. Testing the model.

Task-wise Explanation

a) Setting up the Environment

The TicTacToe class represents the environment. A nine-element list represents the board. The values 0, 1 and −1 represent empty cells, X and O respectively. The reset(), available_actions(), step() and render() methods provide the basic environment interface.

b) Defining the Tic-Tac-Toe Game

The game contains eight possible winning lines: three rows, three columns and two diagonals. The check_winner() method checks these lines and also detects a draw when no empty cell remains.

c) Building the Reinforcement Learning Model

The QLearningAgent class maintains a Q-table. Each board configuration is treated as a state and each board position is an action. The choose_action() method implements ε-greedy exploration, while update() applies the Q-learning update rule.

d) Training the Model

The train_agent() function plays many simulated games. The agent selects legal moves, receives rewards, updates Q-values and gradually reduces ε. In the supplied implementation, 50,000 training episodes are used.

e) Testing the Model

The test_agent() function uses the trained Q-table without exploration and evaluates the agent over multiple games. The opponent in this basic implementation makes random legal moves. Wins, draws and losses are counted and displayed.

Conclusion:

Post Lab Questions:

1. What is reinforcement learning?
2. Define state, action, reward and policy in Tic-Tac-Toe.
3. What is Q-learning?
4. Explain the Q-learning update equation.
5. What is exploration versus exploitation?
6. What is ε-greedy action selection?

Program:

```
# Experiment No. 6: Tic-Tac-Toe using Reinforcement Learning
# Q-Learning based implementation

import random
import numpy as np

class TicTacToe:
    def __init__(self):
        self.reset()

    def reset(self):
        self.board = [0] * 9   # 0 = empty, 1 = X, -1 = O
        self.done = False
        return self.get_state()

    def get_state(self):
        return tuple(self.board)

    def available_actions(self):
        return [i for i, value in enumerate(self.board) if value == 0]

    def check_winner(self):
        winning_lines = [
            (0,1,2), (3,4,5), (6,7,8),
            (0,3,6), (1,4,7), (2,5,8),
            (0,4,8), (2,4,6)
        ]

        for a, b, c in winning_lines:
            s = self.board[a] + self.board[b] + self.board[c]
            if s == 3:
                return 1
            if s == -3:
                return -1

        if 0 not in self.board:
            return 0   # draw

        return None   # game continues

    def step(self, action, player=1):
        if self.done or action not in self.available_actions():
            return self.get_state(), -10, True

        self.board[action] = player
        result = self.check_winner()

        if result is not None:
```

```
            self.done = True
            if result == player:
                return self.get_state(), 1, True
            elif result == 0:
                return self.get_state(), 0, True
            else:
                return self.get_state(), -1, True

        return self.get_state(), 0, False

    def render(self):
        symbols = {1: "X", -1: "O", 0: " "}
        for r in range(3):
            print(
                f" {symbols[self.board[3*r]]} |"
                f" {symbols[self.board[3*r+1]]} |"
                f" {symbols[self.board[3*r+2]]} "
            )
            if r < 2:
                print("---+---+---")


class QLearningAgent:
    def __init__(
        self,
        alpha=0.1,
        gamma=0.9,
        epsilon=1.0,
        epsilon_decay=0.995,
        epsilon_min=0.05
    ):
        self.q_table = {}
        self.alpha = alpha
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_decay = epsilon_decay
        self.epsilon_min = epsilon_min

    def get_q_values(self, state):
        if state not in self.q_table:
            self.q_table[state] = np.zeros(9)
        return self.q_table[state]

    def choose_action(self, state, available_actions, training=True):
        q_values = self.get_q_values(state)

        if training and random.random() < self.epsilon:
            return random.choice(available_actions)

        # Choose the best legal action
```

```
        legal_q = [(a, q_values[a]) for a in available_actions]
        max_q = max(value for _, value in legal_q)
        best_actions = [a for a, value in legal_q if value == max_q]
        return random.choice(best_actions)

    def update(self, state, action, reward, next_state, done):
        current_q = self.get_q_values(state)[action]

        if done:
            target = reward
        else:
            next_q = self.get_q_values(next_state)
            target = reward + self.gamma * np.max(next_q)

        self.q_table[state][action] += self.alpha * (target - current_q)

    def decay_epsilon(self):
        self.epsilon = max(
            self.epsilon_min,
            self.epsilon * self.epsilon_decay
        )


def train_agent(episodes=50000):
    agent = QLearningAgent()

    for episode in range(episodes):
        env = TicTacToe()
        state = env.reset()

        while not env.done:
            actions = env.available_actions()
            action = agent.choose_action(
                state, actions, training=True
            )

            next_state, reward, done = env.step(action, player=1)
            agent.update(
                state, action, reward, next_state, done
            )

            state = next_state

            if done:
                break

            # Opponent: random legal move
            opponent_actions = env.available_actions()

            if opponent_actions:
```

```
            opponent_action = random.choice(opponent_actions)
            next_state, opponent_reward, done = env.step(
                opponent_action, player=-1
            )

            # From the agent's perspective, opponent win is negative
            if done and opponent_reward == 1:
                opponent_reward = -1

            agent.update(
                state, action, opponent_reward,
                next_state, done
            )

            state = next_state

        agent.decay_epsilon()

    return agent


def test_agent(agent, games=10):
    wins = draws = losses = 0

    for game in range(games):
        env = TicTacToe()
        state = env.reset()

        print(f"\nTest Game {game + 1}")
        env.render()

        while not env.done:
            # Agent plays X
            actions = env.available_actions()
            action = agent.choose_action(
                state, actions, training=False
            )
            state, reward, done = env.step(action, player=1)
            env.render()

            if done:
                break

            # Human/random opponent can be replaced by input()
            actions = env.available_actions()
            opponent_action = random.choice(actions)
            state, opponent_reward, done = env.step(
                opponent_action, player=-1
            )
            env.render()
```

```
        result = env.check_winner()

        if result == 1:
            wins += 1
            print("Result: Agent Wins")
        elif result == -1:
            losses += 1
            print("Result: Agent Loses")
        else:
            draws += 1
            print("Result: Draw")

    print("\nTesting Summary")
    print("Wins  :", wins)
    print("Draws :", draws)
    print("Losses:", losses)


# Train the agent
agent = train_agent(episodes=50000)

print("Training completed.")
print("Number of learned states:", len(agent.q_table))
print("Final epsilon:", agent.epsilon)

# Test the trained agent
test_agent(agent, games=10)
```